Unidad 12 · Transacciones, índices y la base dentro de un programa

Qué vas a poder hacer. Agrupar las sentencias de una operación del negocio en una transacción que se confirma entera o se deshace entera, reconocer qué pasa cuando falla a la mitad y cuando dos personas escriben a la vez, decidir qué índice crear leyendo el plan de una consulta, y usar la base desde Python con consultas parametrizadas, sin abrirle la puerta a la inyección SQL.

Lo que necesitas antes. La unidad 5 (claves primarias y foráneas, `CHECK`, `NOT NULL`), la 8 (`INSERT`, `UPDATE`, `DELETE`), la 10 (`JOIN`) y la 11 (la vista `v_ventas_tienda`). De Python basta saber escribir una función, un `for` y un `try`/`except`.

Tiempo. Unas 7 horas: 3 de lectura con SQLite y Python abiertos y 4 de práctica.

Cómo correr todo. Los scripts están en `sql/`. `u12_transacciones.sql` crea la base del ejemplo y es **una sesión**: hay que correrla entera y en orden, porque cada paso depende del anterior y algunas sentencias fallan a propósito. `u12_caso.sql` es la sesión sobre una **copia** de `datos/estampa_sur.db` (registra pedidos y crea índices). `u12_programa.py` tiene todas las funciones de Python de la unidad. Cada resultado es el que devolvieron SQLite 3.50.4 y Python 3.14; el verificador `verificacion/u12_transacciones.py` los vuelve a obtener sobre copias y los busca en el texto. Este ramo no corrió MySQL: donde MySQL difiere, se dice, pero sus salidas no se muestran.

---

1. La idea: una operación del negocio son varias sentencias

Vender dos martillos y cinco cintas en una ferretería no es una sentencia. Es registrar la venta, registrar cada línea y descontar el stock de cada producto: cinco sentencias. Si el programa se cae después de la tercera, la base queda con una venta que dice dos martillos y un stock que solo bajó para uno de los productos. Ninguna regla de integridad lo detecta: cada fila, por separado, es válida.

Una **transacción** es un grupo de sentencias que la base trata como una sola unidad: o quedan todas o no queda ninguna. Las tablas pasan de un estado correcto (antes de la venta) a otro estado correcto (después), sin que nadie vea nunca el estado intermedio.

La base del ejemplo. Una ferretería de Los Ángeles registra ventas y lleva el stock en la misma base. El `CHECK (stock >= 0)` es la regla que va a hacer fallar una venta a propósito.

```
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE producto (
    id_producto INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL,
    precio      INTEGER NOT NULL CHECK (precio > 0),
    stock       INTEGER NOT NULL CHECK (stock >= 0)
);
CREATE TABLE venta (
    id_venta INTEGER PRIMARY KEY,
    fecha    DATE NOT NULL,
    vendedor VARCHAR(30) NOT NULL
);
CREATE TABLE detalle_venta (
    id_venta    INTEGER NOT NULL REFERENCES venta(id_venta),
    id_producto INTEGER NOT NULL REFERENCES producto(id_producto),
    cantidad    INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unit INTEGER NOT NULL,
    PRIMARY KEY (id_venta, id_producto)
);
```

```
SELECT * FROM producto;
```

id_producto	nombre	precio	stock
1	Tornillo volcanita 6x1 (caja 100)	2990	40
2	Martillo carpintero 16 oz	8490	12
3	Cinta aisladora	1290	60
4	Brocha 2 pulgadas	1890	25
5	Silicona transparente	4590	8
6	Guantes de trabajo	3290	30

```
SELECT * FROM detalle_venta;
```

id_venta	id_producto	cantidad	precio_unit
1	1	2	2990
1	3	4	1290
2	2	1	8490
2	6	2	3290
3	5	1	4590
3	4	3	1890
4	1	5	2990
5	3	10	1290
5	6	1	3290

Las ventas 1 a 5 son de Rosa y Tomás entre el 05-10-2026 y el 08-10-2026; el stock de la tabla ya descuenta lo que vendieron.

`PRAGMA foreign_keys = ON` **en SQLite.** SQLite **no revisa las claves foráneas** si no se le pide, y el pedido vale para cada conexión: hay que repetirlo cada vez que se abre la base. MySQL (con InnoDB) las revisa siempre.

---

2. Las propiedades ACID

Lo que una transacción garantiza se resume en cuatro letras:

Letra	Propiedad	Qué garantiza	En la ferretería
A	**Atomicidad**	todo o nada	no queda una venta con la mitad de sus líneas
C	**Consistencia**	al terminar se cumplen todas las reglas (`CHECK`, claves, `NOT NULL`)	ninguna transacción confirmada deja un stock negativo
I	**Aislamiento** (*isolation*)	las transacciones simultáneas no se pisan; cada una ve un estado coherente	la caja 2 no ve la venta a medio registrar de la caja 1
D	**Durabilidad**	lo confirmado sobrevive a un corte de luz	tras el `COMMIT`, la venta está en el disco

La intuición detrás de la A y la D. El motor no escribe directo sobre los datos «buenos»: anota lo que va a cambiar en un diario (*journal* en SQLite, *redo log* y *undo log* en MySQL). Si la transacción se confirma, el diario dice qué quedó; si se deshace o se corta la luz, el diario permite volver atrás. Por eso deshacer es barato y confirmar es seguro.

La C no es solo de la base. La base garantiza las reglas que se le escribieron. «La venta descuenta el stock de cada producto vendido» no está en ningún `CHECK`: la garantiza el programa que pone las cinco sentencias dentro de la misma transacción.

---

3. `BEGIN`, `COMMIT` y `ROLLBACK`

Sin transacción explícita, **cada sentencia es su propia transacción** y se confirma sola al terminar (*autocommit*). Para agrupar, se abre con `BEGIN`, se confirma con `COMMIT` y se deshace con `ROLLBACK`.

Una venta que sale bien. Tomás vende 2 martillos y 5 cintas:

```
BEGIN;
```

```
INSERT INTO venta VALUES (6, '2026-10-09', 'Tomás');
```

```
INSERT INTO detalle_venta VALUES (6, 2, 2, 8490);
```

```
UPDATE producto SET stock = stock - 2 WHERE id_producto = 2;
```

```
INSERT INTO detalle_venta VALUES (6, 3, 5, 1290);
```

```
UPDATE producto SET stock = stock - 5 WHERE id_producto = 3;
```

```
COMMIT;
```

```
SELECT id_producto, nombre, stock FROM producto WHERE id_producto IN (2, 3);
```

id_producto	nombre	stock
2	Martillo carpintero 16 oz	10
3	Cinta aisladora	55

Una venta que falla a la mitad. Rosa vende 3 pares de guantes y 10 siliconas, pero quedan 8 siliconas:

```
BEGIN;
```

```
INSERT INTO venta VALUES (7, '2026-10-09', 'Rosa');
```

```
INSERT INTO detalle_venta VALUES (7, 6, 3, 3290);
```

```
UPDATE producto SET stock = stock - 3 WHERE id_producto = 6;
```

```
INSERT INTO detalle_venta VALUES (7, 5, 10, 4590);
```

```
UPDATE producto SET stock = stock - 10 WHERE id_producto = 5;
```

```
CHECK constraint failed: stock >= 0
```

Lo importante: el error no deshizo la transacción. Deshizo **solo la sentencia que falló**. Las cuatro anteriores siguen ahí, sin confirmar, y la transacción sigue abierta. Mirando desde adentro:

```
SELECT d.id_venta, d.id_producto, d.cantidad, p.stock
FROM detalle_venta d JOIN producto p ON p.id_producto = d.id_producto
WHERE d.id_venta = 7;
```

id_venta	id_producto	cantidad	stock
7	5	10	8
7	6	3	27

Media venta: los guantes ya se descontaron (27) y hay una línea que dice 10 siliconas con el stock de silicona intacto (8). Si ahora alguien escribe `COMMIT`, ese estado queda guardado (ejercicio 12.5). Lo que corresponde es deshacer:

```
ROLLBACK;
```

```
SELECT (SELECT COUNT(*) FROM venta WHERE id_venta = 7) AS ventas_7,
       (SELECT stock FROM producto WHERE id_producto = 6) AS stock_guantes,
       (SELECT stock FROM producto WHERE id_producto = 5) AS stock_silicona;
```

ventas_7	stock_guantes	stock_silicona
0	30	8

La venta 7 no existe y los guantes volvieron a 30. La base quedó exactamente como estaba antes del `BEGIN`.

La regla. Un error dentro de una transacción no la cierra: **quien abrió la transacción decide**. Desde la consola lo decides tú; desde un programa, el programa tiene que atrapar el error y hacer `ROLLBACK` (sección 6). MySQL se comporta igual para los errores de restricción: deshace la sentencia y deja la transacción abierta.

Otros errores que también detienen una sentencia. Una línea con un producto que no existe choca con la clave foránea (con `PRAGMA foreign_keys = ON`):

```
INSERT INTO detalle_venta VALUES (6, 9, 1, 1000);
```

```
FOREIGN KEY constraint failed
```

Diferencias con MySQL.

- Se abre con `START TRANSACTION` (o `BEGIN`). El *autocommit* viene encendido (`autocommit = 1`).
- Las transacciones solo existen con el motor de tablas **InnoDB**, que es el de por omisión. Una tabla MyISAM ignora el `ROLLBACK`.
- Un `CREATE`, `ALTER` o `DROP` dentro de una transacción la **confirma en el acto** (*commit implícito*). En SQLite el DDL también se puede deshacer.
- MySQL respeta los `CHECK` desde la versión 8.0.16; antes los aceptaba y los ignoraba.
- `SAVEPOINT nombre` y `ROLLBACK TO nombre` (en los dos motores) deshacen hasta un punto intermedio sin abandonar la transacción. Sirven en procesos largos; esta unidad no los necesita.

---

4. Concurrencia y aislamiento, como intuición

Una base la usan varias personas a la vez: dos cajas de la ferretería, el sistema de despacho, el informe que corre el gerente. El aislamiento decide **qué ve cada una de lo que hacen las otras** mientras sus transacciones están abiertas.

Lo que puede salir mal.

Anomalía	Qué pasa	Ejemplo
lectura sucia	leo un cambio que el otro todavía no confirma (y quizá deshaga)	la caja 2 ve 27 guantes de la venta 7, que después se deshizo
lectura no repetible	leo la misma fila dos veces en mi transacción y cambió entre medio	el informe lee el stock de silicona al principio y al final y no calza
fantasma	repito una consulta con condición y aparecen filas nuevas	«ventas de hoy» da 6 y, un momento después, 7
actualización perdida	dos leen el mismo valor, cada uno calcula y escribe; gana el último	ver abajo

Los niveles de aislamiento del estándar SQL permiten más o menos de lo anterior. Más aislamiento es más seguro y más lento, porque hay que esperar más:

Nivel	Lectura sucia	No repetible	Fantasma
`READ UNCOMMITTED`	puede pasar	puede pasar	puede pasar
`READ COMMITTED`	no	puede pasar	puede pasar
`REPEATABLE READ`	no	no	puede pasar (InnoDB las evita en las lecturas comunes)
`SERIALIZABLE`	no	no	no

MySQL (InnoDB) usa `REPEATABLE READ` por omisión y se cambia con `SET TRANSACTION ISOLATION LEVEL READ COMMITTED;` (o el nivel que se quiera). **SQLite** es serializable: deja leer a muchos al mismo tiempo, pero escribe **uno a la vez**. Se ve con dos conexiones a la misma base, la caja A y la caja B:

```
def dos_cajas(ruta):
    """La caja A está a mitad de una transacción; la caja B mira y trata de escribir."""
    a = sqlite3.connect(ruta, isolation_level=None)
    b = sqlite3.connect(ruta, isolation_level=None, timeout=0)
    sql = "SELECT stock FROM producto WHERE id_producto = 2"
    visto = []
    a.execute("BEGIN")
    a.execute("UPDATE producto SET stock = stock - 1 WHERE id_producto = 2")
    visto.append(("A, dentro de su transacción", a.execute(sql).fetchall()[0][0]))
    visto.append(("B, mientras A no confirma", b.execute(sql).fetchall()[0][0]))
    try:
        b.execute("UPDATE producto SET stock = stock - 1 WHERE id_producto = 2")
    except sqlite3.OperationalError as e:
        visto.append(("B trata de escribir", str(e)))
    a.execute("COMMIT")
    visto.append(("B, después del COMMIT de A", b.execute(sql).fetchall()[0][0]))
    a.close(); b.close()
    return visto
```

Corrida sobre la base recién creada (12 martillos):

```
A, dentro de su transacción: 11
B, mientras A no confirma: 12
B trata de escribir: database is locked
B, después del COMMIT de A: 11
```

A ve su propio cambio; B no lo ve hasta el `COMMIT` (no hay lectura sucia). B no puede escribir mientras A tiene la transacción abierta: con `timeout=0` falla en el acto; con el valor por omisión (5 segundos) espera y reintenta. Por eso **una transacción debe ser corta**: nunca se deja abierta esperando que el usuario apriete un botón.

La actualización perdida no la evita el motor: la evita cómo escribes. Dos vendedores venden 5 siliconas a la vez. Cada programa lee el stock, calcula en Python y escribe el resultado:

```
def leer_y_escribir(ruta, id_producto, cantidad):
    """Dos vendedores leen el stock, calculan en Python y escriben el resultado (MAL)."""
    a, b = abrir(ruta), abrir(ruta)
    sql = "SELECT stock FROM producto WHERE id_producto = ?"
    stock_a = a.execute(sql, (id_producto,)).fetchall()[0][0]
    stock_b = b.execute(sql, (id_producto,)).fetchall()[0][0]
    vendidas = 0
    for con, visto in ((a, stock_a), (b, stock_b)):
        if visto >= cantidad:
            con.execute("UPDATE producto SET stock = ? WHERE id_producto = ?",
                        (visto - cantidad, id_producto))
            con.commit()
            vendidas += cantidad
    final = a.execute(sql, (id_producto,)).fetchall()[0][0]
    a.close(); b.close()
    return stock_a, vendidas, final
```

```
>>> leer_y_escribir(ruta, 5, 5)
(8, 10, 3)
```

Había 8 siliconas, se vendieron 10 y el stock dice 3. Los dos leyeron 8, los dos vieron que alcanzaba y los dos escribieron 3: la venta del primero se perdió. Ninguna regla falló, porque 3 es un stock válido. El arreglo es que **la condición y el descuento vayan en la misma sentencia**, y que la base los haga juntos:

```
def descontar_en_la_base(ruta, id_producto, cantidad):
    """Los mismos dos vendedores, pero el descuento y la condición van en una sola
sentencia."""
    a, b = abrir(ruta), abrir(ruta)
    filas = []
    for con in (a, b):
        cur = con.execute("UPDATE producto SET stock = stock - ? "
                          "WHERE id_producto = ? AND stock >= ?",
                          (cantidad, id_producto, cantidad))
        con.commit()
        filas.append(cur.rowcount)
    sql = "SELECT stock FROM producto WHERE id_producto = ?"
    final = a.execute(sql, (id_producto,)).fetchall()[0][0]
    a.close(); b.close()
    return filas, final
```

```
>>> descontar_en_la_base(ruta, 5, 5)
([1, 0], 3)
```

El primer `UPDATE` cambió 1 fila; el segundo, 0: cuando le tocó, el stock ya era 3 y la condición `stock >= 5` no se cumplió. `rowcount` le dice al programa que la segunda venta no se hizo y que tiene que avisarle al vendedor. En MySQL, cuando de verdad hay que leer antes de escribir, se usa `SELECT ... FOR UPDATE`, que bloquea la fila hasta el `COMMIT`; en SQLite, `BEGIN IMMEDIATE`, que toma el permiso de escritura desde el comienzo.

---

5. Índices

La idea. Para encontrar las ventas de silicona, el motor recorre toda la tabla `detalle_venta` fila por fila. Un **índice** es una estructura aparte, ordenada por una o más columnas (un árbol B), que le dice dónde están las filas con un valor dado, como el índice alfabético al final de un libro. Con 9 filas no importa; con 2 millones, recorrer todo puede tomar segundos y buscar en el índice, milisegundos.

Cómo saber si una consulta usa un índice. `EXPLAIN QUERY PLAN` delante de la consulta muestra el plan sin ejecutarla. `SCAN` es recorrer la tabla completa; `SEARCH ... USING INDEX` es ir directo.

```
EXPLAIN QUERY PLAN
SELECT SUM(cantidad) FROM detalle_venta WHERE id_producto = 5;
```

```
QUERY PLAN
`--SCAN detalle_venta
```

`detalle_venta` ya tiene un índice: el de su clave primaria `(id_venta, id_producto)`, que SQLite crea solo y llama `sqlite_autoindex_detalle_venta_1`. Pero ese índice está ordenado **primero por** `id_venta`: sirve para buscar por venta, no por producto, igual que una guía telefónica ordenada por apellido y nombre no sirve para buscar a todos los «Rosa».

```
EXPLAIN QUERY PLAN
SELECT SUM(cantidad) FROM detalle_venta WHERE id_venta = 5;
```

```
QUERY PLAN
`--SEARCH detalle_venta USING INDEX sqlite_autoindex_detalle_venta_1 (id_venta=?)
```

Para buscar por producto hace falta un índice que empiece por `id_producto`:

```
CREATE INDEX idx_detalle_producto ON detalle_venta(id_producto);
```

```
EXPLAIN QUERY PLAN
SELECT SUM(cantidad) FROM detalle_venta WHERE id_producto = 5;
```

```
QUERY PLAN
`--SEARCH detalle_venta USING INDEX idx_detalle_producto (id_producto=?)
```

```
SELECT SUM(cantidad) FROM detalle_venta WHERE id_producto = 5;
```

SUM(cantidad)
1

El resultado es el mismo con o sin índice (una silicona vendida, en la venta 3): **un índice nunca cambia la respuesta, solo el camino**.

Una función sobre la columna anula el índice. Con un índice en `venta(fecha)`, un rango se busca:

```
CREATE INDEX idx_venta_fecha ON venta(fecha);
```

```
EXPLAIN QUERY PLAN
SELECT COUNT(*) FROM venta WHERE fecha BETWEEN '2026-10-05' AND '2026-10-07';
```

```
QUERY PLAN
`--SEARCH venta USING COVERING INDEX idx_venta_fecha (fecha>? AND fecha<?)
```

Pero si la condición transforma la columna, el índice ya no sabe dónde buscar y hay que recorrerlo entero (`SCAN`):

```
EXPLAIN QUERY PLAN
SELECT COUNT(*) FROM venta WHERE substr(fecha, 9, 2) = '05';
```

```
QUERY PLAN
`--SCAN venta USING COVERING INDEX idx_venta_fecha
```

Covering (que cubre) significa que el índice tiene todas las columnas que la consulta necesita, así que el motor ni siquiera abre la tabla. Aun así, `SCAN` recorre todo.

Cuándo ayuda un índice.

- Columnas de `WHERE` con igualdad o rango que dejan pocas filas (un RUT, un id de cliente, un rango de fechas corto).
- Claves foráneas por las que se hacen `JOIN`. La clave primaria ya tiene índice; **la foránea, en SQLite, no** (MySQL crea uno al declarar la `FOREIGN KEY`).
- Columnas de `ORDER BY` o `GROUP BY` frecuentes, para evitar ordenar en una tabla temporal (`USE TEMP B-TREE`).

Cuándo estorba o no sirve.

- **Cada escritura paga.** Un `INSERT`, `UPDATE` o `DELETE` tiene que actualizar la tabla **y cada índice**. Una tabla con diez índices escribe once veces.
- Ocupa espacio en disco.
- Tablas chicas: recorrer 200 filas es más rápido que consultar un índice.
- Columnas con pocos valores distintos (un estado con tres valores, un sí o no): el índice apunta a un tercio de la tabla y no ahorra nada.
- La consulta tiene que leer toda la tabla igual (un total general).

- Funciones sobre la columna (`substr(fecha, ...)`, `lower(nombre)`) o un `LIKE '%texto'` que empieza con comodín.

En MySQL, `EXPLAIN SELECT ...` devuelve una tabla con una fila por tabla de la consulta. Las columnas que hay que mirar: `type` (`ALL` es recorrer todo; `ref`, `eq_ref` o `range` es usar un índice), `key` (el índice elegido, o `NULL`) y `rows` (cuántas filas estima leer). `EXPLAIN ANALYZE` (desde la 8.0.18) además ejecuta la consulta y muestra los tiempos reales. El `CREATE INDEX` es igual en los dos motores.

---

6. La base dentro de un programa: Python y `sqlite3`

Nadie registra ventas escribiendo SQL en una consola: lo hace el programa de la caja. Python trae el módulo `sqlite3`; para MySQL se instala un conector (`mysql-connector-python`) que se usa casi igual.

```
import sqlite3
```

```
def abrir(ruta):
    con = sqlite3.connect(ruta)
    con.execute("PRAGMA foreign_keys = ON")
    return con
```

```
def stock_bajo(con, minimo):
    cur = con.cursor()
    cur.execute("SELECT nombre, stock FROM producto WHERE stock < ? ORDER BY stock",
(minimo,))
    return cur.fetchall()
```

```
>>> con = abrir("ferreteria.db")
>>> stock_bajo(con, 30)
[('Silicona transparente', 8), ('Martillo carpintero 16 oz', 12), ('Brocha 2 pulgadas',
25)]
```

(Corrido sobre la base recién creada, antes de la venta 6.)

Pieza	Qué es
`sqlite3.connect(ruta)`	abre la base (la crea si no existe) y devuelve la **conexión**
`con.cursor()`	un **cursor**: el que ejecuta y recorre resultados; `con.execute(...)` crea uno por ti
`cur.execute(sql, parametros)`	ejecuta **una** sentencia; cada `?` se reemplaza por un parámetro
`cur.fetchall()` · `cur.fetchone()`	trae todas las filas, como lista de tuplas · trae la siguiente
`cur.rowcount`	cuántas filas cambió el último `INSERT`, `UPDATE` o `DELETE`
`cur.lastrowid`	el id que se le asignó a la última fila insertada
`con.commit()` · `con.rollback()`	confirma · deshace la transacción abierta
`with con:`	al salir del bloque hace `commit()`; si hubo una excepción, `rollback()`
`con.close()`	cierra; **lo no confirmado se pierde**

Las transacciones en `sqlite3`. Con la configuración por omisión, Python abre solo una transacción antes de cada `INSERT`, `UPDATE` o `DELETE`, y la deja abierta hasta que el programa llama a `commit()` o `rollback()`. Si olvidas el `commit()`, los cambios se pierden al cerrar. La forma segura es `with con:`, que confirma o deshace por ti. Con `isolation_level=None` (como en `dos_cajas`) Python no abre nada solo y tú escribes `BEGIN` y `COMMIT`.

Los parámetros (`?`). `(minimo,)` es una tupla de un elemento (la coma es obligatoria). El valor viaja **aparte** del texto del SQL: la base recibe la consulta con un hueco y el valor para el hueco. Nunca se pega el valor dentro del texto. La sección 7 muestra por qué.

En MySQL el conector usa `%s` en vez de `?` (`cur.execute("... WHERE stock < %s", (minimo,))`) y también necesita `con.commit()`.

---

7. Inyección SQL: por qué nunca se arma el SQL pegando texto

Una búsqueda por nombre escrita pegando el texto del usuario dentro del SQL:

```
def buscar_inseguro(con, texto):
    sql = "SELECT nombre, precio FROM producto WHERE nombre = '" + texto + "'"
    return con.execute(sql).fetchall()
```

Con un nombre normal funciona:

```
>>> buscar_inseguro(con, "Cinta aisladora")
[('Cinta aisladora', 1290)]
```

Ahora alguien escribe en el buscador `x' OR '1'='1`. El texto se pega y la consulta que llega a la base es `SELECT nombre, precio FROM producto WHERE nombre = 'x' OR '1'='1'`: la comilla del usuario cerró la del programa, y la condición `'1'='1'` es verdadera para toda fila.

```
>>> buscar_inseguro(con, "x' OR '1'='1")
[('Tornillo volcanita 6x1 (caja 100)', 2990), ('Martillo carpintero 16 oz', 8490), ('Cinta
aisladora', 1290), ('Brocha 2 pulgadas', 1890), ('Silicona transparente', 4590), ('Guantes
de trabajo', 3290)]
```

Acá «solo» se ve el catálogo completo. En una tabla de clientes son los RUT y teléfonos de todos; en un `UPDATE` o un `DELETE`, son todas las filas (ejercicio 12.8 y caso). Eso es la **inyección SQL**: el dato del usuario se convierte en código.

El arreglo: parámetros.

```
def buscar(con, texto):
    sql = "SELECT nombre, precio FROM producto WHERE nombre = ?"
    return con.execute(sql, (texto,)).fetchall()
```

```
>>> buscar(con, "x' OR '1'='1")
[]
```

Con `?`, el texto completo `x' OR '1'='1` se busca como un nombre, con comillas y todo. No hay ningún producto que se llame así: lista vacía. La base nunca lo interpreta como SQL.

¿Y `'; DROP TABLE producto; --`? El método `execute` de `sqlite3` acepta **una sola** sentencia, así que el ataque clásico de agregar una segunda se detiene:

```
>>> buscar_inseguro(con, "x'; DROP TABLE producto; --")
ProgrammingError: You can only execute one statement at a time.
```

No es una defensa: `executescript` sí ejecuta varias, otros conectores también, y la inyección de una sola sentencia (como `OR '1'='1'`) igual funciona. La única defensa de verdad son los parámetros.

Hazlo	No lo hagas
`con.execute("... WHERE id = ?", (valor,))`	`"... WHERE id = " + valor`
`con.execute("... LIKE ?", ("%" + texto + "%",))`	`f"... LIKE '%{texto}%'"`
validar que un id sea número **además** de usar `?`	creer que escapar comillas a mano basta

Los parámetros reemplazan **valores**, no nombres de tablas ni de columnas. Si el programa tiene que elegir una columna (para ordenar, por ejemplo), la elige de una lista cerrada escrita en el programa, nunca del texto del usuario.

---

8. Ejemplo resuelto · La venta desde la caja

El problema. La ferretería quiere que la caja registre una venta con una sola llamada: se le pasan el número de venta, la fecha, el vendedor y las líneas `(producto, cantidad)`. Si cualquier línea no se puede vender, no se registra nada y el vendedor recibe el motivo.

Paso 1 · En palabras. Dentro de una transacción: insertar la venta; por cada línea, insertar el detalle con el precio vigente del producto y descontar el stock. Si algo falla, deshacer todo. Los datos del usuario van siempre como parámetros.

Paso 2 · El código.

```
def vender(con, id_venta, fecha, vendedor, lineas):
    """Registra una venta con sus líneas y descuenta el stock: todo o nada.
    lineas es una lista de (id_producto, cantidad)."""
    with con:
        con.execute("INSERT INTO venta VALUES (?, ?, ?)", (id_venta, fecha, vendedor))
        for id_producto, cantidad in lineas:
            cur = con.execute("INSERT INTO detalle_venta "
                              "SELECT ?, id_producto, ?, precio FROM producto "
                              "WHERE id_producto = ?",
                              (id_venta, cantidad, id_producto))
            if cur.rowcount != 1:
                raise ValueError(f"no existe el producto {id_producto}")
            con.execute("UPDATE producto SET stock = stock - ? WHERE id_producto = ?",
                        (cantidad, id_producto))
```

Tres decisiones del código, línea por línea:

- `with con:` es la transacción. Si el bloque termina normal, confirma; si cualquier línea lanza una excepción, deshace todo y deja pasar la excepción hacia quien llamó.
- `INSERT ... SELECT` toma el precio de la tabla `producto` en vez de recibirlo del usuario: la caja no puede inventar un precio.
- `rowcount != 1`. Si el producto no existe, `INSERT ... SELECT` inserta **cero filas sin error** (el `SELECT` no encontró nada que insertar). El programa tiene que revisarlo y lanzar el error él mismo; si no, la venta se registraría sin esa línea.

Paso 3 · Una venta que sale bien (sobre la base recién creada): la venta 6 de la sección 3, ahora desde Python.

```
>>> vender(con, 6, "2026-10-09", "Tomás", [(2, 2), (3, 5)])
>>> con.execute("SELECT id_producto, stock FROM producto WHERE id_producto IN (2,
3)").fetchall()
[(2, 10), (3, 55)]
```

Paso 4 · Una venta que no alcanza. Diez siliconas, con 8 en stock:

```
>>> vender(con, 7, "2026-10-09", "Rosa", [(6, 3), (5, 10)])
IntegrityError: CHECK constraint failed: stock >= 0
>>> con.execute("SELECT (SELECT COUNT(*) FROM venta WHERE id_venta = 7), (SELECT stock FROM
producto WHERE id_producto = 6)").fetchall()
[(0, 30)]
```

Paso 5 · Un producto que no existe.

```
>>> vender(con, 7, "2026-10-09", "Rosa", [(6, 3), (9, 1)])
ValueError: no existe el producto 9
>>> con.execute("SELECT (SELECT COUNT(*) FROM venta WHERE id_venta = 7), (SELECT stock FROM
producto WHERE id_producto = 6)").fetchall()
[(0, 30)]
```

En los dos fracasos no hay venta 7 y los guantes siguen en 30: lo que alcanzó a hacerse antes del error (la venta, la línea de guantes, su descuento) se deshizo.

Cómo se lee.

- **La caja muestra el motivo**, que viene en la excepción: «no alcanza el stock» o «no existe el producto 9». El vendedor corrige y vuelve a intentar; nunca queda media venta que alguien tenga que encontrar y arreglar a mano.
- **El `CHECK (stock >= 0)` trabaja para el programa.** Sin esa regla, la venta de 10 siliconas habría pasado y el stock quedaría en −2. La regla está en la base, así que protege también a cualquier otro programa que escriba en ella.
- **Lo que falta para dos cajas a la vez** está en la sección 4: el descuento ya va en una sola sentencia (`stock = stock - ?`), así que dos cajas no se pisan; si se quisiera avisar antes de intentar, se agregaría `AND stock >= ?` y se revisaría `rowcount`.

9. Errores típicos

Error	Cómo evitarlo
Creer que un error dentro de la transacción la deshace sola, y hacer `COMMIT` igual	un error deshace solo esa sentencia; quien abrió la transacción hace `ROLLBACK` (o usa `with con:`)
Sentencias de una misma operación sin transacción (cada una se confirma sola)	`BEGIN` … `COMMIT`, o `with con:` en Python
Olvidar `con.commit()` en Python: los cambios desaparecen al cerrar	`with con:`; revisa con otra conexión que el dato quedó
Olvidar `PRAGMA foreign_keys = ON` en SQLite: las claves foráneas no se revisan	ponlo al abrir cada conexión (función `abrir`)
Leer, calcular en el programa y escribir el resultado (actualización perdida)	`UPDATE ... SET stock = stock - ? WHERE ... AND stock >= ?` y revisar `rowcount`
Dejar una transacción abierta esperando al usuario: los demás quedan bloqueados	la transacción empieza cuando ya están todos los datos y termina enseguida
Armar el SQL pegando texto del usuario (inyección SQL)	parámetros `?` (o `%s` en MySQL) para todo valor
Indexar todas las columnas «por si acaso»	cada índice encarece cada escritura; indexa lo que el plan muestra como `SCAN` en consultas frecuentes
Esperar que un índice compuesto `(a, b)` sirva para buscar por `b`	sirve para `a` o para `a` y `b`; para `b` sola hace falta otro índice
Una función sobre la columna indexada (`substr(fecha, ...)`)	escribe la condición sobre la columna tal cual: un rango `BETWEEN`
`INSERT ... SELECT` con una condición que no encuentra filas: inserta cero sin error	revisa `rowcount`

10. Práctica

Diez ejercicios: 12.1 a 12.4 básicos, 12.5 a 12.8 intermedios, 12.9 y 12.10 de desafío. Los que dicen «ferretería» siguen la sesión de `u12_transacciones.sql` (después de la venta 6 y de los dos índices de la sección 5); los que dicen «caso» usan la copia de la base del caso **después** de la sección 12 (122 pedidos, con los índices de clientes y pedidos ya creados). Las pautas están en la sección 11.

12.1 · ¿Qué letra de ACID? (básico). Di qué propiedad protege (o falta) en cada situación. (a) Se corta la luz un segundo después de que la caja mostró «venta registrada» y, al volver, la venta está. (b) Un programa que traspasa saldo entre dos cuentas se cae entre el abono y el cargo, y al volver ninguna de las dos cambió. (c) Mientras el gerente corre el informe de ventas del día, una caja registra una venta; el informe muestra el total de antes o el de después, pero nunca uno a medias. (d) Una transacción que dejaría un stock negativo no se puede confirmar.

12.2 · Predice y corre (básico, ferretería). Predice qué muestran las dos consultas y después córrelo:

```
BEGIN;
```

```
UPDATE producto SET precio = precio + 100;
```

```
DELETE FROM detalle_venta WHERE id_venta = 1;
```

```
SELECT (SELECT COUNT(*) FROM detalle_venta) AS lineas,
       (SELECT precio FROM producto WHERE id_producto = 1) AS precio_tornillo;
```

```
ROLLBACK;
```

y de nuevo la misma consulta.

12.3 · Lee dos planes (básico, caso). Corre `EXPLAIN QUERY PLAN` de estas dos consultas sobre `detalle_pedido` y explica por qué una busca y la otra recorre, si las dos filtran por una columna de la clave primaria:

```
EXPLAIN QUERY PLAN
SELECT * FROM detalle_pedido WHERE id_pedido = 40;
```

```
EXPLAIN QUERY PLAN
SELECT * FROM detalle_pedido WHERE id_producto = 3;
```

12.4 · La clave foránea que no se revisa (básico, ferretería). (a) ¿Qué responde SQLite a `INSERT INTO detalle_venta VALUES (6, 9, 1, 1000);` con `PRAGMA foreign_keys = ON`, y por qué? (b) ¿Qué pasaría en una conexión abierta sin ese `PRAGMA`? (c) ¿Por qué no basta con ponerlo una vez en el script que creó la base?

12.5 · `COMMIT` después del error (intermedio, ferretería). Repite la venta 7 de la sección 3 (guantes y siliconas), pero cuando el último `UPDATE` falle escribe `COMMIT` en vez de `ROLLBACK`. ¿Qué queda en la base? ¿Qué reglas del negocio quedan rotas aunque ninguna restricción haya fallado? ¿Cómo se arregla después?

12.6 · ¿Qué índice crearías? (intermedio, caso). Estampa Sur corre muy seguido dos consultas: «los pedidos de un cliente» (`WHERE id_cliente = 12`) y «los pedidos pendientes» (`WHERE estado = 'pendiente'`). Mira el plan de cada una y cuántos pedidos hay por estado. ¿Conviene un índice nuevo para alguna? Argumenta con el plan y con los conteos.

12.7 · Dos vendedores y los guantes (intermedio, ferretería). Sobre la base recién creada (30 pares de guantes), dos vendedores venden 20 pares cada uno al mismo tiempo. (a) Predice qué devuelve `leer_y_escribir(ruta, 6, 20)` (stock leído, pares vendidos, stock final) y qué anda mal. (b) Predice `descontar_en_la_base(ruta, 6, 20)`. (c) ¿Qué tiene que hacer el programa del segundo vendedor con lo que le devuelve?

12.8 · Anular una venta (intermedio, ferretería). La pantalla «anular venta» recibe el número como texto y llama a esta función:

```
def anular_venta_inseguro(con, id_texto):
    with con:
        cur = con.execute("DELETE FROM detalle_venta WHERE id_venta = " + id_texto)
    return cur.rowcount
```

(a) ¿Qué pasa si alguien escribe `3 OR 1=1`? (b) Reescríbela con parámetros y di qué devuelve con esa misma entrada. Pruébalo **en una copia de juguete**, nunca en la base de verdad.

12.9 · Traspasos en un curso (desafío). Un colegio lleva los fondos de los cursos en una tabla `cuenta` con `CHECK (saldo >= 0)`: el 4.° A tiene $85.000, el 4.° B $42.500 y el centro de padres $120.000. (a) Escribe `transferir(con, origen, destino, monto)` que abone y después descuente, todo o nada. (b) Pruébala con $50.000 del 4.° B al 4.° A y con $30.000 del centro de padres al 4.° B: ¿cómo quedan los saldos y el total? (c) Corre la misma idea **sin transacción** (cada sentencia se confirma sola) traspasando $80.000 del 4.° B al 4.° A. ¿Qué pasa con el total? (d) ¿Qué invariante sirve para detectar el problema?

12.10 · Ventas de la segunda quincena (desafío, caso). El dueño pide seguido las ventas entre el 16-03-2026 y el 31-03-2026. (a) Mira el plan de la consulta con `p.fecha BETWEEN '2026-03-16' AND '2026-03-31'`. (b) Crea un índice que la ayude y mira el plan de nuevo. (c) Un compañero propone filtrar con `substr(p.fecha, 9, 2) >= '16'`: ¿usa el índice? (d) ¿Cuánto vendió esa quincena? (e) Lista los índices que quedaron en la base y di cuántos índices tiene que actualizar el `INSERT` de un pedido nuevo.

---

11. Pautas

12.1. (a) **Durabilidad**: lo confirmado sobrevive al corte. (b) **Atomicidad**: el traspaso se hizo entero o nada; como no terminó, nada. (c) **Aislamiento**: el informe no ve la venta a medio registrar. (d) **Consistencia**: ninguna transacción confirmada deja la base contra sus reglas.

12.2. Dentro de la transacción se ven los cambios propios: había 11 líneas (las 9 originales y las 2 de la venta 6) y se borraron las 2 de la venta 1; el tornillo sube a $3.090:

lineas	precio_tornillo
9	3090

Después del `ROLLBACK`, todo vuelve (11 líneas, contando las dos de la venta 6, y el precio original):

```
SELECT (SELECT COUNT(*) FROM detalle_venta) AS lineas,
       (SELECT precio FROM producto WHERE id_producto = 1) AS precio_tornillo;
```

lineas	precio_tornillo
11	2990

Ojo con el `UPDATE` sin `WHERE`: cambia **todos** los precios. Dentro de una transacción se deshace; fuera de ella, no hay vuelta atrás.

12.3.

```
QUERY PLAN
`--SEARCH detalle_pedido USING INDEX sqlite_autoindex_detalle_pedido_1 (id_pedido=?)
```

```
QUERY PLAN
`--SCAN detalle_pedido
```

La clave primaria es `(id_pedido, id_producto)` y su índice está ordenado primero por `id_pedido`. Buscar por `id_pedido` es buscar por la primera columna: sirve. Buscar solo por `id_producto` es buscar por la segunda: las filas de un producto están repartidas por todo el índice, y el motor recorre la tabla.

12.4. (a)

```
FOREIGN KEY constraint failed
```

No existe el producto 9 en `producto`. (b) Sin el `PRAGMA`, SQLite inserta la línea sin reclamar: queda un detalle que apunta a un producto inexistente. (c) Porque en SQLite la revisión de claves foráneas es una opción **de la conexión**, no de la base: cada programa que abre la base tiene que pedirla. Por eso la función `abrir` lo hace siempre.

12.5. La sesión es la de la sección 3, con `COMMIT` al final:

```
BEGIN;
```

```
INSERT INTO venta VALUES (7, '2026-10-09', 'Rosa');
```

```
INSERT INTO detalle_venta VALUES (7, 6, 3, 3290);
```

```
UPDATE producto SET stock = stock - 3 WHERE id_producto = 6;
```

```
INSERT INTO detalle_venta VALUES (7, 5, 10, 4590);
```

```
UPDATE producto SET stock = stock - 10 WHERE id_producto = 5;
```

```
CHECK constraint failed: stock >= 0
```

```
COMMIT;
```

```
SELECT d.id_producto, d.cantidad, p.stock
FROM detalle_venta d JOIN producto p ON p.id_producto = d.id_producto
WHERE d.id_venta = 7;
```

id_producto	cantidad	stock
5	10	8
6	3	27

Queda la venta 7 con sus dos líneas: los guantes descontados (27) y una línea de 10 siliconas con el stock de silicona intacto (8). Toda fila cumple sus restricciones, pero se rompieron dos reglas que no están escritas en la base: «lo vendido se descuenta del stock» y «no se vende lo que no hay». Una vez confirmado, el `ROLLBACK` ya no sirve: hay que escribir sentencias que compensen (borrar la línea de silicona o rebajarla a 8 y descontarlas) en una transacción nueva, y averiguar si al cliente se le cobraron las 10.

12.6. Los pedidos de un cliente ya usan un índice (el que creó el caso integrador):

```
EXPLAIN QUERY PLAN
SELECT * FROM pedido WHERE id_cliente = 12;
```

```
QUERY PLAN
`--SEARCH pedido USING INDEX idx_pedido_cliente (id_cliente=?)
```

Los pendientes recorren la tabla:

```
EXPLAIN QUERY PLAN
SELECT COUNT(*) FROM pedido WHERE estado = 'pendiente';
```

```
QUERY PLAN
`--SCAN pedido
```

```
SELECT estado, COUNT(*) AS pedidos FROM pedido GROUP BY estado ORDER BY pedidos DESC;
```

estado	pedidos
entregado	81
en taller	31
pendiente	10

Con 122 pedidos, recorrer la tabla es instantáneo: no conviene un índice nuevo. Un índice en `estado` tiene tres valores distintos; para «entregado» apuntaría a dos tercios de la tabla y no ahorraría nada. Solo se justificaría con muchos miles de pedidos y si la consulta frecuente fuera justamente la del estado raro («pendiente», el 8 % de los pedidos). El de `id_cliente`, en cambio, sí se justifica: cada cliente tiene pocos pedidos y la consulta es diaria.

12.7. (a)

```
>>> leer_y_escribir(ruta, 6, 20)
(30, 40, 10)
```

Los dos leyeron 30, los dos vieron que alcanzaba, se vendieron 40 pares con 30 en bodega y el stock dice 10. Una venta se perdió en el stock y hay 10 pares vendidos que no existen. (b)

```
>>> descontar_en_la_base(ruta, 6, 20)
([1, 0], 10)
```

El segundo `UPDATE` cambió 0 filas: cuando le tocó quedaban 10 pares. (c) Revisar `rowcount`: si es 0, la venta no se hizo, y el programa avisa al vendedor («quedan 10 pares») en vez de imprimir la boleta.

12.8. (a) La sentencia queda `DELETE FROM detalle_venta WHERE id_venta = 3 OR 1=1`, y `1=1` es verdadero para toda fila. En la copia de juguete (la base recién creada, con 9 líneas) borra todas:

```
>>> anular_venta_inseguro(con, "3 OR 1=1")
9
>>> con.execute("SELECT COUNT(*) FROM detalle_venta").fetchall()
[(0,)]
```

(b)

```
def anular_venta(con, id_texto):
    with con:
        cur = con.execute("DELETE FROM detalle_venta WHERE id_venta = ?", (id_texto,))
    return cur.rowcount
```

```
>>> anular_venta(con, "3 OR 1=1")
0
```

El texto completo se compara con `id_venta` como un valor: ninguna venta se llama «3 OR 1=1» y no se borra nada. Además conviene validar que la entrada sea un número antes de llamar a la base, para darle al usuario un mensaje claro.

12.9. (a) La tabla y la función (en `u12_programa.py`):

```
def crear_cuentas(con):
    with con:
        con.execute("CREATE TABLE cuenta (id INTEGER PRIMARY KEY, titular VARCHAR(30) NOT
NULL, "
                    "saldo INTEGER NOT NULL CHECK (saldo >= 0))")
        con.executemany("INSERT INTO cuenta VALUES (?, ?, ?)",
                        [(1, "4.° A", 85000), (2, "4.° B", 42500),
                         (3, "Centro de padres", 120000)])
```

```
def transferir(con, origen, destino, monto):
    """Abona y después descuenta; si el descuento falla, el abono se deshace."""
    with con:
        con.execute("UPDATE cuenta SET saldo = saldo + ? WHERE id = ?", (monto, destino))
        con.execute("UPDATE cuenta SET saldo = saldo - ? WHERE id = ?", (monto, origen))
```

(b)

```
>>> con.execute("SELECT * FROM cuenta").fetchall()
[(1, '4.° A', 85000), (2, '4.° B', 42500), (3, 'Centro de padres', 120000)]
>>> con.execute("SELECT SUM(saldo) FROM cuenta").fetchall()
[(247500,)]
>>> transferir(con, 2, 1, 50000)
IntegrityError: CHECK constraint failed: saldo >= 0
>>> con.execute("SELECT * FROM cuenta").fetchall()
[(1, '4.° A', 85000), (2, '4.° B', 42500), (3, 'Centro de padres', 120000)]
>>> transferir(con, 3, 2, 30000)
>>> con.execute("SELECT * FROM cuenta").fetchall()
[(1, '4.° A', 85000), (2, '4.° B', 72500), (3, 'Centro de padres', 90000)]
>>> con.execute("SELECT SUM(saldo) FROM cuenta").fetchall()
[(247500,)]
```

El primer traspaso alcanzó a abonar $50.000 al 4.° A antes de que el cargo fallara (el 4.° B tenía $42.500); `with con:` deshizo el abono. El segundo pasó. El total sigue en $247.500. (c) Sin transacción:

```
def transferir_sin_transaccion(ruta, origen, destino, monto):
    con = sqlite3.connect(ruta, isolation_level=None)      # cada sentencia se confirma
sola
    try:
        con.execute("UPDATE cuenta SET saldo = saldo + ? WHERE id = ?", (monto, destino))
        con.execute("UPDATE cuenta SET saldo = saldo - ? WHERE id = ?", (monto, origen))
    finally:
        con.close()
```

```
>>> transferir_sin_transaccion(ruta, 2, 1, 80000)
IntegrityError: CHECK constraint failed: saldo >= 0
>>> con.execute("SELECT * FROM cuenta").fetchall()
[(1, '4.° A', 165000), (2, '4.° B', 72500), (3, 'Centro de padres', 90000)]
>>> con.execute("SELECT SUM(saldo) FROM cuenta").fetchall()
[(327500,)]
```

El abono se confirmó solo y el cargo falló: aparecieron $80.000 de la nada y el total subió a $327.500. Cada fila es válida; la suma no. (d) **La suma de los saldos no cambia con un traspaso.** Revisarla antes y después (o en una conciliación diaria) detecta cualquier traspaso a medias.

12.10. (a) Sin índice en la fecha, el motor recorre el detalle y busca cada pedido por su clave:

```
EXPLAIN QUERY PLAN
SELECT SUM(d.cantidad * d.precio_unit) AS ventas
FROM detalle_pedido d
JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE p.fecha BETWEEN '2026-03-16' AND '2026-03-31';
```

```
QUERY PLAN
|--SCAN d
`--SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
```

(b)

```
CREATE INDEX idx_pedido_fecha ON pedido(fecha);
```

```
EXPLAIN QUERY PLAN
SELECT SUM(d.cantidad * d.precio_unit) AS ventas
FROM detalle_pedido d
JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE p.fecha BETWEEN '2026-03-16' AND '2026-03-31';
```

```
QUERY PLAN
|--SEARCH p USING COVERING INDEX idx_pedido_fecha (fecha>? AND fecha<?)
`--SEARCH d USING INDEX sqlite_autoindex_detalle_pedido_1 (id_pedido=?)
```

El plan se dio vuelta: ahora parte por los pedidos del rango (en el índice de fecha) y, para cada uno, busca sus líneas por la clave primaria del detalle. Ya no recorre nada. (c) No:

```
EXPLAIN QUERY PLAN
SELECT SUM(d.cantidad * d.precio_unit) AS ventas
FROM detalle_pedido d
JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE substr(p.fecha, 9, 2) >= '16';
```

```
QUERY PLAN
|--SCAN d
`--SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
```

Es el plan de antes del índice. Además la condición está mal: toma el día 16 o más de **cualquier** mes (los pedidos de abril también, si caen después del 15). (d)

```
SELECT SUM(d.cantidad * d.precio_unit) AS ventas
FROM detalle_pedido d
JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE p.fecha BETWEEN '2026-03-16' AND '2026-03-31';
```

ventas
18034530

La segunda quincena de marzo vendió $18.034.530. (e)

```
SELECT name AS indice, tbl_name AS tabla
FROM sqlite_master
WHERE type = 'index'
ORDER BY tbl_name, name;
```

indice	tabla
idx_cliente_tienda	cliente
sqlite_autoindex_detalle_pedido_1	detalle_pedido
idx_pedido_cliente	pedido
idx_pedido_fecha	pedido
sqlite_autoindex_producto_1	producto
sqlite_autoindex_suministra_1	suministra
sqlite_autoindex_tienda_1	tienda

`pedido` tiene dos índices propios (`idx_pedido_cliente` e `idx_pedido_fecha`) además de la tabla, que está ordenada por su `INTEGER PRIMARY KEY` y no necesita un índice aparte. Un pedido nuevo escribe en la tabla y en esos dos índices: tres escrituras en vez de una. Con unos pocos pedidos al día es un costo despreciable a cambio de que el informe sea inmediato; en una tabla que recibe miles de inserciones por segundo, cada índice se discute.

---

12. Caso integrador · Estampa Sur registra un pedido

> Este caso sigue a Estampa Sur por todo el ramo. Hasta la unidad 11 la base solo se leyó. Acá se escribe: un pedido nuevo entra como transacción, otro falla a la mitad y no deja rastro, el mismo registro se hace desde Python, y la consulta de ventas por tienda recibe sus índices. Todo corre sobre una copia de `datos/estampa_sur.db` con `sql/u12_caso.sql`.

El punto de partida. Se activan las claves foráneas, se crea la vista de la unidad 11 y se cuenta:

```
PRAGMA foreign_keys = ON;
```

```
CREATE VIEW v_ventas_tienda AS
SELECT t.nombre AS tienda, SUM(d.cantidad * d.precio_unit) AS ventas
FROM detalle_pedido d
JOIN pedido p  ON p.id_pedido = d.id_pedido
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN tienda t  ON t.id_tienda = c.id_tienda
GROUP BY t.nombre;
```

```
SELECT (SELECT COUNT(*) FROM pedido) AS pedidos,
       (SELECT COUNT(*) FROM detalle_pedido) AS lineas,
       (SELECT COUNT(*) FROM pedido WHERE id_cliente = 27) AS pedidos_cliente_27;
```

pedidos	lineas	pedidos_cliente_27
120	212	0

El cliente 27 (una persona de la tienda Lagos) es uno de los cuatro sin pedidos de la unidad 10.

El pedido 121, todo o nada. El cliente 27 encarga 4 poleras básicas y 2 polerones. El precio unitario sale de la tabla `producto`, no se escribe a mano:

```
BEGIN;
```

```
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado) VALUES (121, 27, '2026-04-02',
'pendiente');
```

```
INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, precio_unit)
SELECT 121, id_producto, 4, precio FROM producto WHERE id_producto = 1;
```

```
INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, precio_unit)
SELECT 121, id_producto, 2, precio FROM producto WHERE id_producto = 2;
```

```
COMMIT;
```

```
SELECT id_pedido, id_producto, cantidad, precio_unit, cantidad * precio_unit AS monto
FROM detalle_pedido
WHERE id_pedido = 121;
```

id_pedido	id_producto	cantidad	precio_unit	monto
121	1	4	3990	15960
121	2	2	9890	19780

```
SELECT tienda, ventas FROM v_ventas_tienda ORDER BY ventas DESC;
```

tienda	ventas
Valle	7297650
Centro	6951140
Lagos	6170090
Costa	5922540
Puerto	5534370
Cerro	3883390

El pedido vale $35.740 y Lagos pasa de $6.134.350 a $6.170.090. Nadie tocó la vista: se recalculó sola con el pedido nuevo (unidad 11). Las otras cinco tiendas siguen con las cifras de la unidad 10.

El pedido 122 falla a la mitad. El cliente 28 encarga 3 bolsas y, por un error de digitación, 0 gorros:

```
BEGIN;
```

```
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado) VALUES (122, 28, '2026-04-03',
'pendiente');
```

```
INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, precio_unit)
SELECT 122, id_producto, 3, precio FROM producto WHERE id_producto = 4;
```

```
INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, precio_unit)
SELECT 122, id_producto, 0, precio FROM producto WHERE id_producto = 5;
```

```
CHECK constraint failed: cantidad > 0
```

El `CHECK (cantidad > 0)` del caso (unidad 8) detuvo la línea. Desde adentro, el pedido y la línea de bolsas siguen ahí:

```
SELECT (SELECT COUNT(*) FROM pedido WHERE id_pedido = 122) AS pedido_122,
       (SELECT COUNT(*) FROM detalle_pedido WHERE id_pedido = 122) AS lineas_122;
```

pedido_122	lineas_122
1	1

```
ROLLBACK;
```

```
SELECT (SELECT COUNT(*) FROM pedido) AS pedidos,
       (SELECT COUNT(*) FROM detalle_pedido) AS lineas,
       (SELECT COUNT(*) FROM pedido WHERE id_pedido = 122) AS pedido_122;
```

pedidos	lineas	pedido_122
121	214	0

Quedan los 121 pedidos con sus 214 líneas (las 212 más las 2 del pedido 121); del 122 no queda nada.

Índices para las ventas de una tienda. Primero, la consulta de todas las tiendas:

```
EXPLAIN QUERY PLAN
SELECT tienda, ventas FROM v_ventas_tienda ORDER BY ventas DESC;
```

```
QUERY PLAN
|--CO-ROUTINE v_ventas_tienda
|  |--SCAN d
|  |--SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
|  |--SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
|  |--SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
|  `--USE TEMP B-TREE FOR GROUP BY
|--SCAN v_ventas_tienda
`--USE TEMP B-TREE FOR ORDER BY
```

CO-ROUTINE es la vista calculándose por dentro. Recorre todo el detalle (`SCAN d`) y para cada línea busca su pedido, su cliente y su tienda por clave primaria. Esta consulta **tiene que leer todas las líneas** para sumar las seis tiendas: ningún índice le ahorra eso, y todos los `JOIN` ya van por clave primaria. No hay nada que indexar.

La pregunta frecuente de cada encargada, en cambio, es la de **su** tienda:

```
EXPLAIN QUERY PLAN
SELECT tienda, ventas FROM v_ventas_tienda WHERE tienda = 'Centro';
```

```
QUERY PLAN
|--CO-ROUTINE v_ventas_tienda
|  |--SEARCH t USING COVERING INDEX sqlite_autoindex_tienda_1 (nombre=?)
|  |--SCAN d
|  |--SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
|  `--SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
`--SCAN v_ventas_tienda
```

SQLite metió el filtro dentro de la vista y encuentra la tienda Centro por el índice de su nombre (el `UNIQUE` de la unidad 8), pero después **igual recorre todo el detalle**: para ir de la tienda a sus líneas tendría que ir de `tienda` a `cliente` por `id_tienda` y de `cliente` a `pedido` por `id_cliente`, y esas claves foráneas no tienen índice en SQLite. Un índice para el primer tramo:

```
CREATE INDEX idx_cliente_tienda ON cliente(id_tienda);
```

```
EXPLAIN QUERY PLAN
SELECT tienda, ventas FROM v_ventas_tienda WHERE tienda = 'Centro';
```

```
QUERY PLAN
|--CO-ROUTINE v_ventas_tienda
|  |--SEARCH t USING COVERING INDEX sqlite_autoindex_tienda_1 (nombre=?)
|  |--SCAN d
|  |--SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
|  `--SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
`--SCAN v_ventas_tienda
```

El plan no cambió. De nada sirve llegar rápido a los clientes si de los clientes a los pedidos hay que recorrer. Hace falta el camino completo:

```
CREATE INDEX idx_pedido_cliente ON pedido(id_cliente);
```

```
EXPLAIN QUERY PLAN
SELECT tienda, ventas FROM v_ventas_tienda WHERE tienda = 'Centro';
```

```
QUERY PLAN
|--CO-ROUTINE v_ventas_tienda
|  |--SEARCH t USING COVERING INDEX sqlite_autoindex_tienda_1 (nombre=?)
|  |--SEARCH c USING COVERING INDEX idx_cliente_tienda (id_tienda=?)
|  |--SEARCH p USING COVERING INDEX idx_pedido_cliente (id_cliente=?)
|  `--SEARCH d USING INDEX sqlite_autoindex_detalle_pedido_1 (id_pedido=?)
`--SCAN v_ventas_tienda
```

Ahora el plan **baja por las claves**: la tienda por su nombre, sus clientes por `id_tienda`, los pedidos de cada cliente por `id_cliente` y las líneas de cada pedido por la clave primaria del detalle. Ya no hay ningún `SCAN` de una tabla. Con 214 líneas la diferencia no se mide; con cientos de miles de pedidos, la encargada de

Centro deja de esperar a que se lean las líneas de las otras cinco tiendas. La consulta de todas las tiendas, como se anticipó, sigue igual:

```
EXPLAIN QUERY PLAN
SELECT tienda, ventas FROM v_ventas_tienda ORDER BY ventas DESC;
```

```
QUERY PLAN
|--CO-ROUTINE v_ventas_tienda
|  |--SCAN d
|  |--SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
|  |--SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
|  |--SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
|  `--USE TEMP B-TREE FOR GROUP BY
|--SCAN v_ventas_tienda
`--USE TEMP B-TREE FOR ORDER BY
```

Y la respuesta es la misma de siempre:

```
SELECT tienda, ventas FROM v_ventas_tienda WHERE tienda = 'Centro';
```

tienda	ventas
Centro	6951140

En MySQL estos dos índices no habría que crearlos: InnoDB indexa cada clave foránea al declararla.

El mismo registro desde Python. La tienda no va a escribir `BEGIN` en una consola. La función del caso (en `u12_programa.py`) deja que la base asigne el número de pedido y toma los precios de `producto`:

```
def registrar_pedido(con, id_cliente, fecha, lineas):
    """Registra un pedido de Estampa Sur con su detalle, todo o nada.
    lineas es una lista de (id_producto, cantidad); el precio sale de la tabla producto."""
    with con:
        cur = con.execute("INSERT INTO pedido (id_cliente, fecha, estado) "
                          "VALUES (?, ?, 'pendiente')", (id_cliente, fecha))
        id_pedido = cur.lastrowid
        for id_producto, cantidad in lineas:
            cur = con.execute("INSERT INTO detalle_pedido "
                              "(id_pedido, id_producto, cantidad, precio_unit) "
                              "SELECT ?, id_producto, ?, precio FROM producto "
                              "WHERE id_producto = ?",
                              (id_pedido, cantidad, id_producto))
            if cur.rowcount != 1:
                raise ValueError(f"no existe el producto {id_producto}")
    return id_pedido
```

El pedido del cliente 28, primero con el mismo error y después corregido (1 gorro):

```
>>> registrar_pedido(con, 28, "2026-04-03", [(4, 3), (5, 0)])
IntegrityError: CHECK constraint failed: cantidad > 0
>>> registrar_pedido(con, 28, "2026-04-03", [(4, 3), (5, 1)])
122
```

El primer intento no dejó nada: por eso el segundo recibe el número 122 y no el 123. Al final:

```
SELECT (SELECT COUNT(*) FROM pedido) AS pedidos,
       (SELECT COUNT(*) FROM detalle_pedido) AS lineas,
       (SELECT COUNT(*) FROM cliente c
        WHERE NOT EXISTS (SELECT 1 FROM pedido p WHERE p.id_cliente = c.id_cliente)) AS
clientes_sin_pedidos;
```

pedidos	lineas	clientes_sin_pedidos
122	216	2

```
SELECT tienda, ventas FROM v_ventas_tienda ORDER BY ventas DESC;
```

tienda	ventas
Valle	7308310
Centro	6951140
Lagos	6170090
Costa	5922540
Puerto	5534370
Cerro	3883390

El pedido 122 vale $10.660 (3 bolsas y 1 gorro) y el cliente 28 es de la tienda Valle, que pasa de $7.297.650 a $7.308.310. Quedan 2 clientes sin pedidos (el 29 y el 30), y cada peso nuevo de la vista corresponde exactamente a un pedido confirmado.

La inyección, en una copia de juguete. La pantalla de despacho marca un pedido como entregado a partir del número que escribe el operador. Con el SQL armado pegando texto:

```
def marcar_entregado_inseguro(con, id_texto):
    with con:
        sql = "UPDATE pedido SET estado = 'entregado' WHERE id_pedido = " + id_texto
        cur = con.execute(sql)
    return cur.rowcount
```

```
>>> marcar_entregado_inseguro(con, "5 OR 1=1")
120
>>> sorted(con.execute("SELECT estado, COUNT(*) FROM pedido GROUP BY estado").fetchall())
[('entregado', 120)]
```

Los 120 pedidos de la copia quedaron «entregados», incluidos los 31 que estaban en el taller y los 8 pendientes. Por eso se prueba en una copia de juguete y nunca en la base del caso. Con parámetros, sobre otra copia:

```
def marcar_entregado(con, id_texto):
    with con:
        sql = "UPDATE pedido SET estado = 'entregado' WHERE id_pedido = ?"
        cur = con.execute(sql, (id_texto,))
    return cur.rowcount
```

```
>>> marcar_entregado(con, "5 OR 1=1")
0
>>> marcar_entregado(con, "5")
1
```

La entrada maliciosa no calza con ningún pedido; la normal marca exactamente uno.

La cadena. El verificador crea la base del caso en una carpeta temporal con el mismo script (`datos/estampa_sur.db` no se toca), comprueba que parte de los 120 pedidos, las 212 líneas y las ventas por tienda del caso, corre `u12_caso.sql` y las funciones de Python, y exige que Lagos y Valle suban exactamente lo registrado y las otras cuatro tiendas no se muevan (`CADENA: OK`).

Dónde termina el ramo. Estampa Sur partió en la unidad 1 como una planilla que ya no alcanzaba. Ahora tiene un diagrama, ocho tablas normalizadas con sus claves, consultas que responden las preguntas del dueño, vistas para las frecuentes, pedidos que entran enteros o no entran, índices donde el plan los pide, y un programa que escribe en la base sin abrirle la puerta a nadie.

---

13. Para seguir

En los libros de referencia del área. El programa del ramo no fija bibliografía. Elmasri y Navathe, *Fundamentos de sistemas de bases de datos*, y Silberschatz, Korth y Sudarshan, *Fundamentos de bases de datos*, tienen capítulos completos de transacciones, control de concurrencia, recuperación e indexación, con la teoría que esta unidad deja como intuición (planificaciones serializables, bloqueo en dos fases, árboles B+).

Abiertos y gratis.

- Markus Winand, *Use The Index, Luke!*, libro en línea sobre índices y planes de ejecución, en español, con ejemplos para MySQL, PostgreSQL, Oracle y SQL Server: https://use-the-index-luke.com/es
- Documentación de Python, módulo `sqlite3` (en español): conexiones, cursores, parámetros y transacciones: https://docs.python.org/es/3/library/sqlite3.html
- SQLite, *Transaction* y *EXPLAIN QUERY PLAN* (en inglés): https://www.sqlite.org/lang_transaction.html y https://www.sqlite.org/eqp.html
- OWASP, *SQL Injection Prevention Cheat Sheet* (en inglés), la referencia práctica contra la inyección: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Con el tutor. La carpeta `tutor/` trae el contexto de este ramo. Pégalo al inicio de la conversación con cualquier IA y pídele, por ejemplo: «Dame una función de Python que registre algo en dos tablas con un error de transacción escondido y otro de inyección SQL; no me digas dónde están hasta que yo los encuentre».